

Model Performance & Audit System

ABSTRACT

Machine Learning (ML) projects involve continuous experimentation with different models, datasets, and hyperparameters. Tracking these experiments manually using spreadsheets or text files is a common practice among students and early-stage researchers, but it quickly becomes disorganized, error-prone, and difficult to manage. The **Model Performance and Audit System (MPAS)** is a web-based ML Experiment Tracking platform developed to solve this problem.

MPAS provides a centralized dashboard where users can log their ML experiments, manage different model architectures, upload dataset details, and track specific hyperparameters. The system is built using HTML5, CSS3, and Vanilla JavaScript for the frontend, ensuring a lightweight and responsive user interface. The backend relies on Supabase, a PostgreSQL-based Database-as-a-Service, which communicates with the frontend via REST API.

Key features include a dark-mode enabled UI with glassmorphism styling, real-time analytics dashboards powered by Chart.js for visualizing accuracy, loss trends, and F1-scores, and an interactive leaderboard to rank the best-performing models. By automating the tracking process, MPAS helps students and researchers maintain a clean audit trail of their ML lifecycle, making experimentation more reproducible and organized.

1. INTRODUCTION

1.1 Introduction to ML Experiment Tracking

Machine Learning is highly iterative. A single project might involve training dozens of models using different architectures (like CNNs or Random Forests), various datasets, and tweaking numerous hyperparameters (like learning rate, batch size, or number of epochs). Keeping a record of which combination produced which result is called Experiment Tracking. It is a fundamental practice to ensure that ML research is reproducible.

1.2 Need for MPAS

In a college environment or during initial stages of learning, students often rely on rudimentary methods like writing down results in a notebook, saving multiple versions of Python scripts (e.g., `model_final_v2_real.py`), or using Excel sheets. These methods are inefficient. If a student wants to find out which hyperparameters gave the 95% accuracy two weeks ago, it is often impossible. Therefore, there is a strong need for a simple, accessible, and visual tracking system.

1.3 Problem Statement

To design and develop a web-based Model Performance and Audit System (MPAS) that provides an easy-to-use interface for logging Machine Learning experiments, managing model and dataset metadata, and visually comparing performance metrics to eliminate the disorganization of manual tracking methods.

1.4 Objectives

- To build a web-based dashboard using HTML, CSS, and JavaScript.
- To implement a relational database (PostgreSQL via Supabase) to store experiment data securely.

- To allow users to register models, datasets, and hyperparameters.
 - To provide dynamic charts (using Chart.js) for visualizing accuracy, loss, and F1-scores over time.
 - To create a leaderboard system that automatically ranks the best experiment runs based on selected metrics.
-

2. LITERATURE SURVEY

2.1 Existing Experiment Tracking Systems

There are several industry-grade tools available for ML tracking, such as MLflow, Weights & Biases (W&B), and TensorBoard.

- **TensorBoard:** Primarily used for TensorFlow, good for visualizing neural network graphs but lacks robust tabular experiment comparison out of the box.
- **MLflow:** A heavy, comprehensive platform meant for enterprise-level deployment and MLOps.
- **Weights & Biases:** A cloud-based platform that requires external API integration inside Python scripts.

2.2 Limitations of Manual Tracking

Before implementing automated systems, tracking is done manually. The limitations include:

- **Data Loss:** Spreadsheets can be accidentally deleted or overwritten.
- **Lack of Context:** A recorded accuracy of 92% is useless if the exact learning rate and dataset version used for that run are forgotten.
- **No Visualization:** Staring at rows of numbers makes it hard to identify trends, such as whether a model is overfitting over time.

2.3 Comparison with Existing Tools

While MLflow and W&B are powerful, they are often too complex and heavy for a 2nd-year B.Tech mini-project or for students just starting out. They require setting up complex Python environments.

MPAS is designed to be lightweight. It uses a simple REST API approach where students can manually log their results via a clean web UI. It acts as an introductory bridge to professional MLOps tools without the steep learning curve.

3. PROPOSED SYSTEM

3.1 Overview of MPAS

The proposed system, Model Performance and Audit System (MPAS), is a lightweight, responsive web application. Users interact with a frontend interface built with Vanilla JavaScript and CSS3. They can input data regarding their models, datasets, and the results of their experiment runs. This data is sent via REST API to a Supabase PostgreSQL database. The frontend then fetches this data to populate interactive charts and leaderboards.

3.2 System Workflow

1. **Registration:** The user registers a new Model (e.g., "ResNet50") and a Dataset (e.g., "SkinCancer Images v1").

2. **Experiment Logging:** The user runs their ML code locally in Python/Jupyter. Once they get the results (Accuracy, Loss), they open MPAS and log a new "Run". They select the Model, Dataset, input the metrics, and attach any Hyperparameters used.
3. **Data Storage:** The frontend makes a POST request to Supabase, storing the data in relational tables.
4. **Visualization:** The user navigates to the Analytics section. The JavaScript fetches the updated data and re-renders the Chart.js canvases to show how this new run compares to previous ones.
5. **Leaderboard Ranking:** The leaderboard automatically updates, sorting the runs by Accuracy or F1-score to highlight the best performing model.

3.3 Advantages

- **No Complex Setup:** Runs entirely in the browser using HTML/JS. No heavy Node.js or Python backend servers needed for the UI.
 - **Visual Appeal:** Uses modern UI elements like dark mode and glassmorphism.
 - **Relational Integrity:** Uses PostgreSQL foreign keys to ensure data consistency (e.g., you cannot log a run for a model that doesn't exist).
-

4. SYSTEM ARCHITECTURE

4.1 Frontend Architecture

The frontend is built without heavy frameworks like React or Angular to keep the project simple.

- **HTML5:** Provides the structure (forms, tables, canvas elements).
- **CSS3:** Handles styling, dark mode toggling via CSS variables, and layout using Flexbox/Grid.
- **Vanilla JavaScript:** Handles DOM manipulation, event listeners, form submissions, and asynchronous `fetch` calls to the Supabase API.

4.2 Backend Architecture

The system uses a Backend-as-a-Service (BaaS) architecture provided by **Supabase**. This eliminates the need to write complex server-side routing code (like Express.js or Flask) for basic CRUD operations. Supabase automatically generates RESTful APIs based on the PostgreSQL database schema.

4.3 Database Architecture

The database is a relational **PostgreSQL** instance. It uses strict schema definitions with Primary Keys (PK) and Foreign Keys (FK) to link users to their runs, models to runs, and hyperparameters to runs.

4.4 REST API Flow

1. **Client (Browser):** User clicks "Add Model". JS gathers form data.
2. **Request:** JS uses the `supabase-js` library to send an HTTP POST request containing JSON data to the Supabase endpoint URL using a secure API Key.
3. **Server (Supabase):** Validates the request, inserts the row into the PostgreSQL table.
4. **Response:** Server sends back an HTTP 201 Created status.
5. **Client:** JS alerts "Model Added Successfully!" and updates the UI.

4.5 Deployment Architecture

The frontend files (`index.html` , CSS, JS) are hosted on **Netlify**, a popular static site hosting platform. When a user accesses the Netlify URL, their browser downloads the static files, which then communicate directly with the Supabase database hosted in the cloud.

5. DATABASE DESIGN

5.1 Schema Explanation

The database consists of five main tables designed to capture the entire state of an ML experiment.

1. `users` : Stores basic information about the researcher.
2. `models` : A catalog of different ML algorithms or neural network architectures being tested.
3. `datasets` : Tracks the data used, which is critical because a model's performance changes depending on the dataset version.
4. `experiment_runs` : The central table. It links a user, a model, and a dataset, and stores the resulting performance metrics (accuracy, loss, etc.).
5. `hyperparameters` : Stores key-value pairs (like "epochs: 50") linked to a specific experiment run.

5.2 Table Relationships

- **One-to-Many**: One User can have Many Experiment Runs.
- **One-to-Many**: One Model can be used in Many Experiment Runs.
- **One-to-Many**: One Dataset can be used in Many Experiment Runs.
- **One-to-Many**: One Experiment Run can have Many Hyperparameters.

5.3 Primary and Foreign Keys

Table: `users`

- `user_id` (Primary Key, Integer, Auto-incrementing)
- `username` (VARCHAR)
- `email` (VARCHAR)
- `role` (VARCHAR)
- `team_name` (VARCHAR)

Table: `models`

- `model_id` (Primary Key, Integer)
- `model_name` (VARCHAR)
- `architecture_type` (VARCHAR) - e.g., CNN, RNN
- `framework` (VARCHAR) - e.g., PyTorch, Keras
- `version` (VARCHAR)
- `release_date` (DATE)

Table: `datasets`

- `dataset_id` (Primary Key, Integer)
- `dataset_name` (VARCHAR)
- `version_tag` (VARCHAR)
- `source_link` (VARCHAR)
- `created_at` (TIMESTAMP)

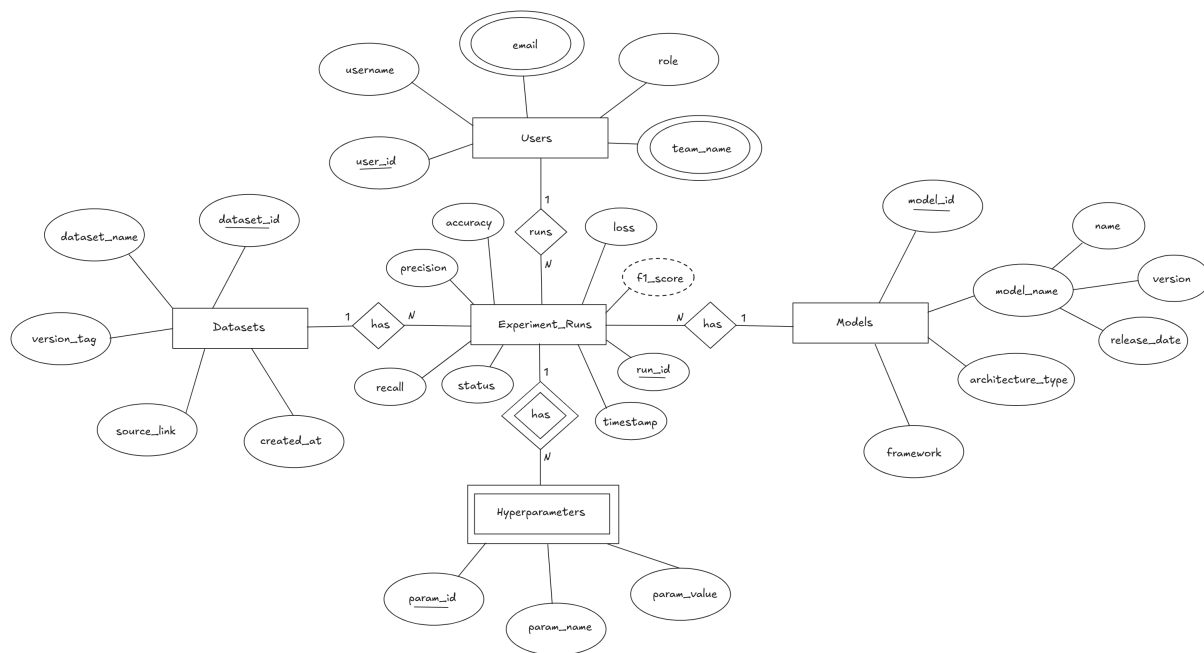
Table: `experiment_runs`

- `run_id` (Primary Key, Integer)
 - `user_id` (Foreign Key referencing `users.user_id`)
 - `dataset_id` (Foreign Key referencing `datasets.dataset_id`)
 - `model_id` (Foreign Key referencing `models.model_id`)
 - `status` (VARCHAR) - e.g., "completed", "failed"
 - `run_timestamp` (TIMESTAMP)
 - `precision_score` (NUMERIC)
 - `recall` (NUMERIC)
 - `accuracy` (NUMERIC)
 - `loss` (NUMERIC)
 - `f1_score` (NUMERIC)
- (Note: Foreign keys use `ON DELETE CASCADE`, meaning if a model is deleted, all its runs are also deleted).

Table: `hyperparameters`

- `param_id` (Primary Key, Integer)
- `run_id` (Foreign Key referencing `experiment_runs.run_id`)
- `param_name` (VARCHAR) - e.g., "learning_rate"
- `param_value` (VARCHAR) - e.g., "0.001"

5.4 ER Diagram Explanation



The Entity-Relationship diagram visually represents the schema. The `experiment_runs` table sits at the center. It has three incoming connections from `users`, `models`, and `datasets`, indicating that a run is a combination of these three entities. It has one outgoing connection to `hyperparameters`, indicating that a run "owns" its parameter settings.

6. DATABASE NORMALIZATION

Database normalization is the process of organizing data to reduce redundancy and improve data integrity. Our schema aims to adhere to standard normal forms.

6.1 First Normal Form (1NF)

A table is in 1NF if it contains no repeating groups or arrays, and every column holds atomic values.

- **Example in MPAS:** Instead of having a single column `hyperparameters` in the `experiment_runs` table storing a comma-separated string like "lr:0.01, batch:32", we created a separate `hyperparameters` table. Each parameter is stored in its own row with an atomic `param_value`.

6.2 Second Normal Form (2NF)

A table is in 2NF if it is in 1NF and all non-key attributes are fully functional dependent on the primary key (no partial dependency).

- **Example in MPAS:** In the `experiment_runs` table, the `accuracy` and `loss` depend entirely on the specific `run_id`. We do not store the `model_name` inside `experiment_runs`; we only store the `model_id`. The `model_name` depends on `model_id`, which is defined in the `models` table.

6.3 Third Normal Form (3NF)

A table is in 3NF if it is in 2NF and has no transitive dependencies (non-key attributes should not depend on other non-key attributes).

- **Example in MPAS:** In the `users` table, `username` and `email` depend directly on `user_id`. There are no fields like "Department_Head_Name" that would depend on "Department" rather than the "User".

6.4 Why Normalization is Needed

- **Reduces Redundancy:** If the name of a model changes (e.g., from "ResNet" to "ResNet50"), we only update it once in the `models` table, rather than updating hundreds of rows in the `experiment_runs` table.
 - **Improves Data Integrity:** It prevents anomalies. For instance, we cannot accidentally insert an experiment run for a non-existent model because the Foreign Key constraint will block it.
-

7. MODULES DESCRIPTION

7.1 User Management

Handles the identity of the person logging the experiment. Currently implemented as a lightweight user tagging system where a `user_id` is assigned to distinguish between different team members in the `users` table.

7.2 Experiment Logging

This is the core module. It provides a form where users can select a pre-registered model and dataset from dropdown menus, and input the results of their testing phase: Accuracy, Loss, Precision, Recall, and F1-Score. It timestamps the run automatically.

7.3 Dataset Upload

Users can register datasets manually via a form or use the **JSON Dataset Upload** support. The system reads a local JSON file using the HTML5 FileReader API, parses it, and bulk-inserts the dataset details into the database, saving time.

7.4 Model Registry

A catalog module where users define the architectural specifications of the models they plan to use. This prevents spelling mistakes when logging runs (e.g., typing "VGG-16" once vs "VGG16" another time).

7.5 Analytics Dashboard

A visual module built using Chart.js. It fetches the data from the `experiment_runs` table and renders it onto HTML Canvas elements. It provides a quick visual summary of whether models are improving over time.

7.6 Leaderboard

A tabular display that aggregates the runs and presents them in a clean table format. It allows users to quickly scan and identify which specific `run_id` achieved the highest F1-score or lowest loss.

7.7 Hyperparameter Tracking

A secondary logging module linked to the main experiment runs. Users input the `run_id` and the specific parameters they tweaked (like learning rate or optimizer type) so they can reproduce the exact conditions of successful runs.

8. UI/UX DESIGN

The user interface was designed to look modern and professional, moving away from basic, unstyled HTML forms typical of early college projects.

8.1 Dark Mode Interface

The application features a system-wide Dark Mode. This is implemented using a CSS class (`.dark-mode`) toggled on the `<body>` tag. When active, CSS variables switch the background to deep blue/grey gradients and change text colors to light grey, reducing eye strain for developers working late.

8.2 Glassmorphism Design

The input cards and modal boxes utilize subtle shadows, rounded borders, and semi-transparent backgrounds to create a "glass-like" depth effect over the background gradients. This gives the application a premium feel.

8.3 Responsive Layout

The dashboard uses CSS Flexbox (`display: flex; flex-wrap: wrap;`). This ensures that on large screens, the input forms sit side-by-side, but on smaller laptop screens or mobile devices, they stack vertically to maintain readability.

8.4 Robot Animation Section

To make the UI engaging, an HTML5 `<canvas>` element is used to render a "Robot-themed animated hero section". This section utilizes JavaScript to draw shapes or load sequences that create a dynamic visual effect when the user first loads the page, establishing a "tech-focused" theme.

8.5 Interactive Dashboard

Forms feature hover effects (`transform: translateY(-5px)`) and focus states with glowing borders (`box-shadow`) to provide immediate visual feedback when the user interacts with the system.

9. DATA VISUALIZATION

Raw tabular data is difficult to interpret. MPAS integrates **Chart.js**, a popular open-source JavaScript charting library, to turn database rows into actionable insights.

9.1 Accuracy Comparison Chart

A grouped **Bar Chart** is used to display the "Metrics Comparison". For each experiment run, it plots two bars side-by-side: Accuracy (blue) and F1-Score (green). This allows the user to instantly see if a model with high accuracy is actually suffering from a low F1-score (which indicates poor performance on imbalanced data).

9.2 F1-Score Chart

Included within the grouped bar chart, the F1-score is plotted alongside accuracy. It is a critical metric in ML for classification tasks, and visualizing it helps ensure models are robust, not just accurate.

9.3 Loss Trend Line Graph

A **Line Chart** is used to plot the "Loss Trend Over Runs". The X-axis represents sequential runs, and the Y-axis represents the loss value. The area under the line is filled with a translucent red color. A downward slope in this graph visually confirms that the model architecture tweaks are successfully reducing error.

9.4 Dynamic Filtering

The analytics dashboard includes dropdown menus to filter the charts. Users can select "All Datasets" or filter to see the charts only for a specific dataset or specific model. When a dropdown changes, a JavaScript function triggers, filters the JSON array in memory, and re-renders the Chart.js instances seamlessly without a page reload.

\pagebreak

10. IMPLEMENTATION DETAILS

10.1 Frontend Implementation

The frontend is written in a single `index.html` file (for simplicity in this mini-project). The CSS is housed within a `<style>` block, and the logic is within a `<script>` block.

- The UI uses basic HTML inputs: `<input type="text">`, `<select>`, `<button>`.
- The dark mode is triggered by a graphical "lightbulb" element using an `onclick` event listener.

10.2 API Integration

We use the `@supabase/supabase-js` CDN library.

```
const supabaseUrl = 'https://[PROJECT-ID].supabase.co';
const supabaseKey = '[API-KEY]';
const sb = supabase.createClient(supabaseUrl, supabaseKey);
```

This initializes the connection securely.

10.3 Database Queries

Data fetching is done asynchronously using JavaScript `async/await`.

```
// Example of fetching models
async function loadData() {
  const responseModels = await sb.from('models').select('*');
  if (responseModels.data) models = responseModels.data;
}
```

Inserting data uses the `.insert()` method:

```
// Example of logging a run
const { error } = await sb.from('experiment_runs').insert([
  { model_id: 1, dataset_id: 1, accuracy: 0.95 }
]);
```

10.4 Chart Rendering

Chart.js is initialized by getting the 2D context of a canvas:

```
const ctx = document.getElementById('accuracyChart').getContext('2d');
myChart1 = new Chart(ctx, { type: 'bar', data: { ... } });
```

When filters are applied, the old chart instance is destroyed (`myChart1.destroy()`) before rendering the new one to prevent glitching.

10.5 Form Validation

Basic frontend validation is implemented. If a user tries to log a run without selecting a model, an `alert()` prevents the API call:

```
if (!run_id.value || !model_select.value) return alert("Fill required fields");
```

11. SCRIPTS & UTILITIES

To manage the backend effectively, three utility scripts were developed.

11.1 restore_supabase.py

A Python script designed to restore database records from a local backup file (`database_backup.json`).

- **Function:** It reads the JSON file and uses the `urllib.request` library to make HTTP POST requests to the Supabase REST API.
- **Logic:** It carefully inserts data in a specific order (users → models → datasets → experiment_runs → hyperparameters) to ensure that Foreign Key constraints are not violated during the restoration process.

11.2 setup_new_supabase.sql

A SQL script used to initialize the database schema.

- **Function:** It contains `CREATE TABLE` statements for all five tables.
- **Features:** It defines Primary Keys (`GENERATED BY DEFAULT AS IDENTITY`), Foreign Keys with `ON DELETE CASCADE` , and disables Row Level Security (`DISABLE ROW LEVEL SECURITY`) to allow the public-facing dashboard to read data without complex authentication for this project scope.

11.3 ping_database.bat

A simple Windows batch script used for network diagnostics.

- **Function:** It helps the student developers quickly verify if their local machine has outbound connectivity to the Supabase cloud servers before troubleshooting complex API issues.

12. TESTING

Testing ensures the application behaves as expected. For this mini-project, functional testing was prioritized.

12.1 Functional Testing

We tested the core functionalities: adding a model, adding a dataset, logging a run, and checking if the charts update correctly.

12.2 Sample Test Cases

Test Case ID	Description	Input	Expected Output	Status
TC_01	Add new model	Name: ResNet, Type: CNN	Model saved to DB, dropdown updates	Pass
TC_02	Add run missing model	Run ID: 1, Acc: 0.9	Alert: "Please fill required fields"	Pass
TC_03	JSON Dataset Upload	Valid JSON file	Datasets added, dropdown updates	Pass
TC_04	Toggle Dark Mode	Click lightbulb	UI background darkens, chart text turns white	Pass
TC_05	Chart Filtering	Select specific Model	Charts re-render showing only that model's data	Pass

12.3 Expected vs Actual Output

- **Expected:** When a new run is logged with an accuracy of 0.98, the Bar Chart should immediately reflect a new bar reaching near the top of the Y-axis.
- **Actual:** Upon clicking "Run", the data is sent to Supabase. The `updateTable()` function triggers, fetching the new data, and the chart re-renders successfully showing the new 0.98 bar.

13. ADVANTAGES & LIMITATIONS

Advantages

1. **Centralized Organization:** Eliminates the chaos of using multiple spreadsheets and text files to track research.
2. **Visual Insights:** Chart.js integration makes it incredibly easy to spot if a model is overfitting (loss going up while accuracy stays high).
3. **Lightweight:** Because it uses Vanilla JS and a BaaS (Supabase), there is no heavy local server to configure. It runs directly from the browser.
4. **Data Integrity:** Relational database constraints ensure that experiment logs are always tied to valid models and datasets.

Limitations

1. **Manual Entry:** Currently, users must manually type in the accuracy and loss values from their Python terminal into the web UI. It does not automatically hook into Python code like enterprise tools.
2. **No Authentication:** To keep the project scope manageable, Row Level Security is disabled, meaning anyone with the link can view or edit the data.
3. **Scalability:** While Supabase is highly scalable, the frontend fetches all runs to render charts. If there are thousands of runs, the browser might slow down. Pagination is not yet implemented.

14. FUTURE SCOPE

The MPAS project lays a strong foundation for a robust MLOps tool. Future enhancements could include:

1. **Python SDK Integration:** Developing a small Python package (`import mpas`) that students can use inside their Jupyter notebooks to automatically send metrics to the Supabase backend via API, eliminating manual data entry.

2. **User Authentication:** Implementing JWT-based login so different teams or students can have private workspaces for their experiments.
 3. **Export Functionality:** Adding a button to export the leaderboard and run data back into a CSV or PDF format for final project submissions.
 4. **Pagination and Lazy Loading:** Improving frontend performance by loading experiment runs in chunks of 50 rather than fetching the entire database at once.
-

15. CONCLUSION

The **Model Performance and Audit System (MPAS)** successfully addresses the common problem of disorganized experiment tracking faced by students and novice researchers in Machine Learning. By combining a responsive, modern web interface with a robust PostgreSQL backend, the project demonstrates how software engineering principles can be applied to improve data science workflows.

The system allows users to meticulously log their models, datasets, and hyperparameters, and instantly visualizes the results through interactive charts. Implementing this project provided valuable hands-on experience in full-stack web development, REST API integration, and relational database design. MPAS serves as a practical, implementable, and highly useful tool that bridges the gap between chaotic manual tracking and complex enterprise MLOps platforms.

16. REFERENCES

- [1] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th ed. Pearson, 2015. (For Database Normalization and Schema Design concepts).
- [2] Mozilla Developer Network (MDN), "HTML: HyperText Markup Language," [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/HTML>.
- [3] Mozilla Developer Network (MDN), "JavaScript," [Online]. Available: <https://developer.mozilla.org/en-US/docs/Web/JavaScript>.
- [4] Chart.js Documentation, "Simple HTML5 Charts using the <canvas> tag," [Online]. Available: <https://www.chartjs.org/docs/latest/>.
- [5] Supabase Documentation, "The Open Source Firebase Alternative," [Online]. Available: <https://supabase.com/docs>.
- [6] I. Sommerville, *Software Engineering*, 10th ed. Pearson, 2015. (For Software Requirements and System Workflow design).